

3. Bridge Design Pattern

- Decouple an abstraction from its implementation so that the two can vary independently.

In simple words

- Separate what the system does
- From how it does it.

Why Bridge Exists:-

⇒ Inheritance Explosion.

Imagine this requirement:

shape: circle, square.
color: Red, Blue.

Red Circle
Blue Circle
Red Square
Blue Square

- Abstraction hierarchy - what the client uses.
- Implementation hierarchy - how it is done.
- They are connected via composition, not inheritance.

Implementor Interface

interface Color {

void applyColor();

Concrete Implementor

class RedColor implements Color {

public void applyColor() {

s.o.pln("Red");

class BlueColor implements Color {

public void applyColor() {

s.o.pln("Blue");

Abstraction

abstract class Shape {

~~protected class class~~
protected Color color;

Shape (Color color) {

 this.color = color;

 abstract void draw();

}

Redefine Abstraction.

class Circle extends Shape {

 Circle (Color color) {

 super (color);

 void draw() {

 System.out.println ("Circle with ");

 color.apply (color);

 }

class Square extends Shape {

 Square (Color color) {

 super (color);

 void draw() {

 System.out.println ("Square with ");

 color.apply (color);

 }

Client code:

Shape redCircle = new Circle (new Red (color));

Shape blueSquare = new Square (new Blue (color));

redCircle.draw();

blueSquare.draw();

- Bridge handles multiple dimensions of change.

Adapter

used after design

makes incompatible interface work.

Reactive

Bridge

used during design.

separate abstraction & implementation.

proactive.